AI Practice Bot Project Documentation

1. Project Overview

AI Practice Bot is a full-stack AI-powered web application for interview preparation. The system lets users register, log in, select a job role and difficulty, answer interview questions, receive AI-assisted coaching, and save their practice history for review.

2. Objectives

- Provide realistic interview practice for common job roles.
- Use AI as a core feature for question enhancement, feedback, hints, sample answers, follow-up prompts, and session summaries.
- Store authenticated users' practice sessions in Supabase.
- Provide a responsive web interface that works on desktop and smaller screens.
- Prepare the project for public GitHub and Vercel submission.

3. Scope

Included:

- User registration, login, logout, email update, and password update.
- AI-assisted interview practice flow.
- Saved sessions dashboard, session details, update, and delete actions.
- Analytics and profile screens.
- Supabase schema and Row Level Security policies.
- README and documentation for deployment and evaluation.

Not included:

- Real-time voice transcription.
- Admin panel for managing all users.
- Payment or subscription features.

4. Technology Stack

Layer	Technology
Frontend	React, TSX, Vite
Styling	Tailwind CSS
Backend/Database	Supabase Auth and Supabase Postgres
AI	Gemini API
Deployment Target	Vercel
Version Control Target	GitHub public repository

5. System Design

The app is a Vite single-page React application. The frontend manages the interview workflow, calls Gemini for AI coaching, and uses Supabase for authentication and database operations.

High-level flow:

1. User registers or logs in through Supabase Auth.
2. User chooses role, difficulty, and number of questions.
3. App prepares local questions and optionally enhances them with Gemini.
4. User answers questions in a chat-like interface.
5. App gives local feedback immediately and enhances feedback with Gemini when available.
6. Completed sessions are saved to Supabase.
7. User reviews, updates, or deletes saved sessions.

6. Database Schema

sessions

Column Type Purpose

id uuid Primary key generated by the app
user_id uuid References auth.users(id)
role text Interview role selected by the user
difficulty text Easy, Medium, or Hard
total_score integer Total score for the session
accuracy integer Percentage score
followups integer Number of follow-up prompts
created_at timestamptz Session creation timestamp

questions

Column Type Purpose

id uuid Primary key generated by the app
session_id uuid References sessions(id)
question text Interview question
answer text User answer
score integer Question score
feedback text Feedback text
created_at timestamptz Record creation timestamp

Row Level Security limits users to their own sessions and related questions.

7. AI Component Explanation

The AI feature is implemented in src/services/geminiService.js. Gemini is used for:

- Generating role-specific interview questions.
- Evaluating candidate answers.
- Producing follow-up questions when answers are short or vague.
- Creating hints and topic explanations.
- Producing sample answers.
- Summarizing completed interview sessions.

The app also has local fallback logic in src/utils/hybridInterview.js so the practice flow
remains usable if the Gemini API key is missing, rate limited, or temporarily unavailable.

8. Main Functional Requirements

Requirement Status

User authentication Complete
Responsive UI Complete
AI-powered core feature Complete
Database CRUD operations Complete
Error handling and validation Complete for key flows, with room for further polish
React TypeScript Implemented through TSX React files and gradual TS config

9. Development Process

1. Built the React interface with Vite and Tailwind CSS.
2. Added Supabase authentication for registration and login.
3. Created the interview practice flow with local question data.
4. Integrated Gemini for AI-powered coaching.
5. Added Supabase persistence for sessions and questions.
6. Added dashboard, session management, profile, settings, and security controls.
7. Converted React components to TSX and added TypeScript configuration.
8. Replaced the default README and prepared documentation.
9. Cleaned ESLint configuration and fixed source lint issues.

10. Team Contribution

Replace the placeholder names before final submission.

Role Member Contribution

Leader TODO Managed scope, planning, task assignment, final review
Frontend TODO Implemented React/TSX views and app state flow
Backend TODO Built Supabase auth, database schema, RLS, and CRUD services
AI TODO Designed Gemini prompts and fallback AI logic
UI/UX TODO Designed responsive interface, layout, and user flows
QA/Documentation TODO Tested build/lint, prepared README, PDF docs, and submission checklist

11. Peer Evaluation

Member Role Rating Notes
:
TODO Leader TODO TODO
TODO Frontend TODO TODO
TODO Backend TODO TODO
TODO AI TODO TODO
TODO UI/UX TODO TODO
TODO QA/Documentation TODO TODO

12. AI Use Documentation

AI was used in two ways:

1. Runtime application feature: Gemini powers interview coaching features including

feedback, follow-ups, hints, sample answers, and summaries.
2. Development assistance: AI coding assistance was used to help plan implementation, debug styling and lint issues, draft documentation, and improve code structure.

The team remains responsible for reviewing the final code, testing the application, and verifying that all submitted work is original and properly documented.

13. Deployment Plan

Target platform: Vercel.

Steps:

1. Create a public GitHub repository.
2. Push the final project to GitHub.
3. Import the repository into Vercel.
4. Add environment variables for Gemini and Supabase.
5. Deploy using npm run build and dist.
6. Add the Vercel link to README and documentation.

Vercel link: TODO

GitHub link: TODO

14. Testing and Verification

Local verification commands:

‘‘bash
npm run lint
npm run build
‘‘

Current result:

- Lint passes.
- Production build passes.
- Vite reports a bundle-size warning because the app is currently built as one large chunk. This is a warning, not a build failure.

15. Submission Checklist

Item Status

Vercel link TODO
GitHub public repository link TODO
PDF documentation Prepared locally
Demo video Optional TODO
README Complete, pending final links
Team names and peer evaluation TODO